

超啟發式演算法近五年研究趨勢，以及量子計算與 Hypercubic Encoding 的整合可行性研究

執行摘要

本報告以 2021 年至 2026 年 9 月 28 日為主要研究區間，聚焦超啟發式演算法（hyper-heuristics, HH）在一般組合最佳化與排程問題上的發展，並向外延伸至量子啟發式演算法、QAOA、量子退火（quantum annealing, QA）、量子位元／離散變數編碼，以及 hyper-cubic / hypercubic encoding。2019–2020 年以前的文獻僅在定義、理論基礎或重要編碼技術無法由近五年文獻取代時納入，例如 QAOA、domain-wall encoding 與 HyperNEAT。近年的 HH 綜述顯示，研究重心已明顯從固定規則的 heuristic selection，轉向強化學習、自學習、代理模型、多目標、平行化、演算法自動組態與生成式／LLM 輔助的高階決策器。¹

核心判斷是：「超啟發式 × 量子 × hypercubic」值得研究，但最有研究價值與近期可實作性的方向，不是把完整問題一次丟到量子電腦，而是建立 quantum-aware hyper-heuristic，讓高階控制器依問題狀態選擇 classical、quantum-inspired、QAOA、QA 等低階求解器，並用 Boolean hypercube 的局部子立方體（subcube）作為可控的量子鄰域。這種架構恰好避開目前量子最佳化最大的幾項瓶頸：QUBO 編碼膨脹、量子位元數不足、minor embedding、chain breaks、QAOA 深度與噪聲，以及量子呼叫的額外成本。D-Wave 上的 JSSP 與 RCPSP 實驗均顯示，排程可以轉成 QUBO 並在真實量子退火器上求解，但編碼與硬體規模仍是主要限制；近年的 QAOA 排程實驗也仍多落在小型問題、分解問題或淺電路情境。²

特別重要的是，hypercubic encoding 並不是目前 HH 文獻中一個統一、成熟的專有編碼方法名稱。至少存在三種容易混淆的用法：第一是 $x \in \{0, 1\}^n$ 的 Boolean hypercube 解空間；第二是 HyperNEAT 的 hypercube-based indirect encoding；第三是在量子資訊／量子糾錯領域使用的 hypercube encoding。對本研究最有價值的是第一種，即把組合解表示成超立方體頂點，將 bit-flip、Hamming distance、固定部分變數後形成的 subcube 作為鄰域；HyperNEAT 則可提供「間接編碼／結構生成」上的啟示，但不能直接等同於組合最佳化的 hypercubic encoding。³

量子端最有立即研究價值的編碼並非單一「量子位元編碼」，而是比較 binary、one-hot、domain-wall、QUBO/Ising 與 quantum-inspired probability-qubit。Chancellor 的 domain-wall 研究在合成排程與圖著色問題上發現，其 embedding 表現於所測案例中與 one-hot 相當或更好，但優勢強烈依賴問題結構，這恰好提供一個 HH 的切入點：不要預先固定 encoding，而讓高階 hyper-heuristic 學習「什麼 instance／什麼 search state 適合什麼 encoding + solver」。⁴

本報告因此給出的優先順序為：

優先度	研究方向	近期可行性	潛在學術價值	判斷
A	HH + quantum-inspired LLHs + Boolean-subcube neighborhoods	很高	高	最適合立即開始；完全可在 CPU/GPU 完成
A	Encoding-aware HH：binary / one-hot / domain-wall / Gray-like representation 自動選擇	很高	很高	與現有 HH、量子最佳化之間有明確研究缺口

優先度	研究方向	近期可行性	潛在學術價值	判斷
A-	HH + D-Wave QA / reverse annealing 作為局部改善器	高	很高	QUBO 與 local-search 結構自然，真實 QPU 可驗證
B+	HH + warm-start QAOA 作為 subproblem / neighborhood solver	中	很高	論文潛力高，但硬體噪聲與電路深度風險較高
B	QAOA/QA 直接最佳化 heuristic sequence	中	很高	很新穎，可把 HH 高階控制本身轉為 QUBO
C	將完整 JSSP/RCPSP 一次性編碼至 QAOA/QA	低至中	中	容易遭遇 qubit/constraint explosion；不建議作第一階段主軸

上述判斷與近年量子最佳化實驗吻合：QAOA 有明確的混合量子—經典最佳化結構，warm-start 與 custom mixer 已證明能顯著改善部分 Max-Cut 實驗；D-Wave 的排程研究則證明真實 QA 硬體可工作，但並沒有形成「所有組合最佳化問題普遍超越經典演算法」的證據。⁵

最重要的研究定位應因此是：

不要把題目定義成「用量子電腦取代 hyper-heuristic」；更好的研究問題是：「Hyper-heuristic 能否學習何時、在哪個子空間、用哪一種 encoding 呼叫哪一種 classical/quantum solver？」

這個問題與 HH 原本「跨低階 heuristic 的自動選擇／生成」精神完全一致，也與量子最佳化正在發展的 warm-start、decomposition、encoding selection、hybrid quantum-classical 思路高度契合。⁶

研究範圍與方法論

本研究將「近五年」操作性定義為 **2021–2026 年截至 2026-09-28**；若某項基礎方法在此前提出但仍是目前方法不可省略的起點，例如 Farhi、Goldstone、Gutmann 的 QAOA、Chancellor 的 domain-wall encoding，或 Stanley 等人的 HyperNEAT hypercube-based encoding，則列為「基礎文獻」，而不計作近五年趨勢本身。QAOA 原始論文將組合最佳化問題映射為 cost Hamiltonian，以深度參數 p 控制交替演化層數；近年的 IBM 官方文件仍以 hybrid quantum-classical iterative method 定義 QAOA。⁷

資料來源優先序採以下原則：原始期刊／出版社與論文頁面優先，其次為 IEEE、ACM、Springer、Elsevier/ScienceDirect、Quantum、Scientific Reports 等原始研究；再其次為 arXiv 原始預印本；官方軟體與硬體技術資訊則採 IBM Quantum/Qiskit、Google Quantum AI/Cirq、D-Wave/Ocean、PennyLane 官方文件。Google Scholar 類服務只作 citation chaining／查找用途，不作核心效能證據來源。HH 的總體趨勢則另外納入近年 peer-reviewed survey/systematic review。¹

主要檢索詞群包括：

主題	檢索概念
Hyper-heuristics	hyper-heuristic, hyperheuristic, selection hyper-heuristic, generative hyper-heuristic, heuristic selection, heuristic generation

主題	檢索概念
學習型 HH	reinforcement learning hyper-heuristic, Q-learning, deep reinforcement learning, genetic programming, surrogate, LLM hyper-heuristic
問題類型	job shop scheduling, workflow scheduling, nurse rostering, bin packing, RCPSP, combinatorial optimization
量子最佳化	QAOA, quantum annealing, reverse annealing, QUBO, Ising, hybrid quantum classical optimization
量子啟發	quantum-inspired evolutionary algorithm, QIEA, quantum chromosome, rotation gate, probability amplitude
編碼	qubit encoding, one-hot, binary encoding, domain-wall encoding, Gray code, constraint preserving mixer
Hypercube	hypercube encoding, hypercubic encoding, hypercube-based encoding, Boolean hypercube, Hamming neighborhood

需要特別指出：在本次檢索到的 2021–2026 文獻中，**沒有發現一條成熟研究線明確以「hyper-heuristic + quantum computing + hypercubic encoding」三者同時作為核心架構**。因此後面的三者整合屬於「由既有 HH、量子最佳化及 Boolean-hypercube 文獻合理推導的新研究架構」，而不是既有成熟方法的摘要。這個「未發現」應解讀成研究缺口，而不是證明世界上不存在任何相關工作。

評估時亦將「量子」分成四種不同層次，避免混稱。**Quantum-inspired** 是在經典電腦上模擬量子式表示或更新，不代表使用量子硬體；**QAOA** 是 gate-model 的混合量子—經典變分最佳化；**quantum annealing** 是以 Ising/QUBO 能量最小化為核心的專用量子退火；「simulated quantum annealing」若指以 classical Monte Carlo 模擬 transverse-field annealing，則依然是經典計算。這四類方法的實證意義不能相互替代。QAOA 與 QA 在現有工具鏈中都以 binary/QUBO/Ising representation 為重要介面，而 D-Wave Ocean 明確提供 BQM/QUBO、classical samplers、QPU 與 hybrid solvers 的共同軟體層。⁸

超啟發式演算法近五年研究版圖

近五年的 HH 發展可概括成一個轉變：從「手工規則決定下一個 heuristic」轉向「讓資料驅動的高階控制器學習演算法選擇、組合與生成」。2024 年的 taxonomy survey 將 HH 的核心定位在操作低階 heuristic，而非直接在原始 solution space 中固定執行單一 metaheuristic；近年的 RL-HH 綜述則顯示，Q-learning、value-based RL 以及更複雜的學習機制已成為重要研究支線。⁹

值得注意的另一條發展是「HH 不只是 heuristic selection，而逐漸變成 algorithm design / configuration layer」。Cruz-Duarte 等人使用 HH 為 continuous optimization 自動客製 metaheuristics；Zhong 等人的 surrogate ensemble-assisted HH 同時選擇搜尋策略與 surrogate model；2025 年的 LLMOA 更把大型語言模型放進高階設計迴路，顯示 HH 與 automated algorithm design 的邊界正在模糊。¹⁰

近五年代表性論文與綜述

論文	年份	方法類型	問題／資料	主要貢獻	效能觀察／指標	主要限制
Hyper-heuristics: A survey and taxonomy — Dokeroglu, Kucukyilmaz, Talbi	2024	Survey / taxonomy	HH 全領域	系統整理 selection/generation 等 HH 與 ML、多目標、平行化等方向	文獻分類而非單一 benchmark	不提供單一方法的實驗優越性
A review of reinforcement learning based hyper-heuristics — Li et al.	2024	RL-HH review	多種組合最佳化	系統整理 RL 如何作為 HH heuristic-selection 控制器	比較 value-based 等 RL-HH 分類與應用	RL state/action/reward 設計仍高度 problem-dependent
Hyper-Heuristics to customise metaheuristics for continuous optimisation — Cruz-Duarte et al.	2021	Metaheuristic design HH	Continuous optimization benchmarks	HH 自動組合／客製 metaheuristic 元件，將 HH 推向 automated algorithm design	以 benchmark objective quality、跨問題排名比較	主要為連續最佳化，轉至高維度約束排程仍需重新設計表示與 operator
Q-learning and hyper-heuristic based algorithm recommendation for changing environments — Gölcük, Ozsoydan	2021	Q-learning selection HH	Dynamic/changing optimization environments	用 Q-learning 學習在環境改變時選擇適合的 bio-inspired optimizer	solution quality、不同環境下適應能力／演算法排名	RL policy 對 state representation 與訓練分布敏感
A hierarchical hyper-heuristic for the bin packing problem — Guerriero, Saccomanno	2023 ; online 2022	Hierarchical selection HH	2D irregular bin packing benchmark	以 instance characteristics 與階層式 HH 選擇 low-level heuristics	packing quality 與 benchmark comparison	問題模型含特定幾何假設；作者亦指出旋轉／翻轉等限制
Surrogate Ensemble-Assisted Hyper-Heuristic Algorithm for Expensive Optimization Problems — Zhong et al.	2023	Surrogate + HH	CEC2013 5D/10D/30D；3 個工程問題	建立 exploration、exploitation、surrogate estimation、mutation 等策略 archive，並在 PR/SVR/GPR 等 surrogate 間適應選擇	嚴格 1000 fitness evaluations 預算下比較 solution quality	成效依 surrogate quality；高維與 distribution shift 是挑戰

論文	年份	方法類型	問題／資料	主要貢獻	效能觀察／指標	主要限制
A cooperative coevolutionary genetic programming hyper-heuristic for multi-objective makespan and cost optimization in cloud workflow scheduling — Zaki et al.	2024	GP / cooperative coevolution HH	Cloud workflow scheduling	同時針對 makespan 與 cost 演化 heuristic/program components	makespan、cost 與 multi-objective solution quality	GP 訓練成本、跨 workflow generalization 是實務問題
LLMOA: A novel large language model assisted hyper-heuristic optimization algorithm — Zhong et al.	2025	LLM-assisted HH	CEC2014/2020/2022 ; 10 engineering problems	以 LLM/ prompting 參與高階演算法決策，低階包含多種 search operators	objective quality、跨 benchmark 排名／比較	API/model variability、本、重現性與 prompt dependence
Systematic literature review of multi-objective hyper-heuristics	2026	Systematic review	Multi-objective HH	顯示多目標 HH 已形成獨立而持續擴張的研究線	文獻分類與研究缺口	2026 年的新結果仍應視為快速演化中的證據

綜合這些工作，可以將 2021–2026 HH 的研究軸整理為 **learning-to-select**、**learning-to-generate**、**surrogate-assisted**、**multi-objective**、**distributed/parallel**、**algorithm recommendation**、**automated metaheuristic design**、**LLM-assisted design**。其中與量子整合最直接相關的是前三類：因為一個 quantum solver 完全可以被視為 HH 的 low-level heuristic，而「是否呼叫量子 solver、使用哪一種 encoding、使用多大的量子鄰域」可以由 RL 或 contextual algorithm-selection policy 決定。這是從既有 HH taxonomy 與量子演算法 selection 思路自然延伸出的研究設計。^[20]

量子方法在啟發式與超啟發式中的應用

量子最佳化與 HH 的交集目前仍遠小於「量子最佳化」或「HH」各自的文獻量。因此研究設計不能假設 quantum LLH 一定更強；比較合理的是把 quantum solver 當作具有不同探索偏好的昂貴低階 heuristic，讓 HH 依 context 選擇何時使用它。這與「algorithm selection」本身相容，也允許量子方法失敗時回退到 tabu search、SA、ILS、GA、CP-SAT 等 classical solvers。現有 QAOA 與 QA 實驗都支持這種保守的 hybrid positioning，而不是支持普遍的量子優越性主張。^[21]

QAOA 對二元組合最佳化可寫成

$$|\psi(\gamma, \beta)\rangle = \prod_{l=1}^p e^{-i\beta_l H_M} e^{-i\gamma_l H_C} |+\rangle^{\otimes n},$$

其中 H_C 對應問題成本， H_M 是 mixer Hamiltonian，經典 optimizer 更新 γ_l, β_l 。增加 p 通常增加 expressive power，也會增加電路深度與噪聲負擔，因此「QAOA depth」本身就可以成為 HH 的一個 action/context 變數。²²

QA/QUBO 則通常以

$$\min_{\mathbf{x} \in \{0,1\}^n} E(\mathbf{x}) = \sum_i h_i x_i + \sum_{i < j} Q_{ij} x_i x_j$$

或等價 Ising $s_i \in \{-1, +1\}$ 表達。真正困難的部分常不是 objective 本身，而是把 precedence、resource、permutation、one-machine-at-a-time 等 constraints 轉成 penalty terms，並控制 logical-to-physical embedding 的增長。D-Wave Ocean 的 BQM/QUBO stack 也反映了這種模型。²³

具體量子／量子啟發案例

方法與案例	問題	編碼／硬體	實驗結果或重要發現	對 HH 的意義	限制	原始來源
QAOA — Kurowski et al.	Job Shop Scheduling	JSSP → quantum optimization / QAOA	系統研究 QAOA 適配排程的可能性；實驗以相對小型 JSSP 為主，重點為 feasibility 與參數／mapping 行為，而非證明 industrial quantum advantage	可把 QAOA 當局部 schedule repair / improvement LLH	完整 JSSP 編碼會快速增加 qubits 與 constraints	EJOR 310, 518–528 ; DOI 10.1016/j.ejor.2023.03.013 ²⁴
Warm-start QAOA + custom mixers — Tate et al.	Weighted Max-Cut	SDP warm start + QAOA	以 1,148 個、最多 11 nodes 的 graphs 模擬至 $p = 8$ ；2D/3D warm-start 在 $p = 0$ 具有 0.658/0.585 worst-case bounds；作者亦展示 20-node example 中 ideal warm-start QAOA 於 $p \geq 2$ 超過該例 GW result	強烈支持「HH incumbent → warm-start quantum LLH」	效果依 warm-start、問題類型與硬體噪聲而異	Quantum 7, 1121 (2023) ²⁵

方法與案例	問題	編碼／硬體	實驗結果或重要發現	對 HH 的意義	限制	原始來源
Iterative-QAOA	Just-in-Time JSSP	IonQ Forte QPU；較大案例用 tensor-network simulation	2025 預印本報告其評估的 QPU instances 均取得 optimal 或 high-quality near-optimal solutions，並將模擬擴至 97 qubits	顯示 iterative / warm-start decomposition 比一次性 variational QAOA 更適合作為 HH low-level solver	仍為預印本；不能由此推論對 classical SOTA 的一般 speedup	arXiv: 2510.26859 26
Quantum annealing — Carugno et al.	JSSP	D-Wave Advantage；QUBO + minor embedding	研究 116 個問題 ，分析 sample 數、annealing duration、chain strength、chain breaks、reverse annealing；顯示真實 JSSP 可被映射但小問題也可能需要大量 qubits	QA 可作 neighborhood optimizer；reverse annealing 尤其適合由 incumbent 開始	logical-to-physical qubit 膨脹與 chain breaks 是核心瓶頸	Scientific Reports 12, 6539 (2022) 27
Quantum annealing — Pérez Armas et al.	RCPSP	12 種 MILP formulations → 最省 qubit 版本 → QUBO；D-Wave Advantage 6.3	首先比較 12 個 MILP formulation 的 qubit efficiency；作者報告 small-medium instances 有潛力，並採 time-to-target、Atos Q-score、reverse annealing 等評估	非常適合做「HH 決定何時送 RCPSP subproblem 至 QA」的基準	問題放大後 encoding size 成為主要障礙	Scientific Reports 14, 16784 (2024) 28

方法與案例	問題	編碼／硬體	實驗結果或重要發現	對 HH 的意義	限制	原始來源
Domain-wall quantum encoding — Chancellor	Synthetic scheduling、graph colouring	Ising domain-wall vs one-hot；Chimera/Pegasus embedding	在作者檢查的案例中 domain-wall embedding 均呈現與 one-hot 相當或更佳 的表現，但優勢高度 problem-structure dependent	直接支持「encoding 也應是 hyper-heuristic action」	非所有問題 domain-wall 都優於 one-hot	Quantum Sci. Technol. 4, 045004 ⁴
Quantum-inspired evolutionary representation	Permutation flow-shop	Quantum chromosome / rotation-angle-based decoding，在 classical computer 執行	既有 QDEA 工作以 quantum-inspired chromosome 與 rotation-angle rule 決定 job sequence，證明這類表示可直接用於 scheduling	最容易納入 HH operator pool，完全沒有 QPU 限制	屬「quantum-inspired」，不能稱為量子加速	International Journal of Advanced Manufacturing Technology 原 ²⁹

QAOA 的 warm-start 結果對 HH 尤其重要。傳統 HH 通常手上已經有 incumbent x_t ，因此沒有理由每次讓 QAOA 從完全均勻的 $|+\rangle^{\otimes n}$ 開始；更合理的是將目前解、classical relaxation 或前一輪結果注入 quantum initialization。Qiskit Optimization 也直接提供 warm-start QAOA 教學與 RecursiveMinimumEigenOptimizer 思路，意味「經典 decomposition → 小量子問題」已經是可實作的工具鏈，而非純概念。³⁰

另一方面，QA 對 HH 有一個 QAOA 沒有那麼直接的優點：**reverse annealing 很像一個以 incumbent 為中心的量子局部搜尋器**。JSSP 與 RCPSP 的 D-Wave 工作都實際討論 reverse annealing，因此可以設計 HH action：「classical perturbation → reverse QA improvement → repair/acceptance」。這比嘗試把整個排程問題一次解完更符合現有硬體能力。³¹

至於「量子模擬退火」需要術語分流。若指 **simulated quantum annealing (SQA)**，它本質上是在 classical hardware 上模擬量子退火動態，因此非常適合作為量子硬體前的 low-level heuristic 與 ablation baseline；若指理論上的 gate-model quantum simulated annealing，則屬另一類量子演算法。在本次 2021–2026 的 HH 交叉檢索中，沒有發現 SQA 已形成像 Q-learning HH 一樣成熟的「SQA-based hyper-heuristic」研究線，因此把 SQA 加進 low-level pool 本身就是值得測試的研究缺口，而不宜假稱目前已有充分 HH 實證。

量子啟發式的 qubit chromosome 通常可概念化為

$$q_i = \begin{bmatrix} \alpha_i \\ \beta_i \end{bmatrix}, \quad |\alpha_i|^2 + |\beta_i|^2 = 1,$$

測量後得到 binary variable，而 rotation-like update 改變下一次採樣機率。它真正有價值的地方不是「模擬了量子神奇加速」，而是提供一種**概率式 population representation**；在 HH 裡可用於 solution encoding，

也可以反過來編碼「第 t 步選哪一個 heuristic」。排程中的 quantum-inspired differential evolutionary work 已證明量子式 chromosome/rotation decoding 可以對 permutation schedule 做實際操作，但這仍完全屬經典演算法。²⁹

Hyper-cubic / hypercubic encoding：定義、數學與最佳化應用

「hypercubic encoding」是整個題目中最需要先澄清的部分。檢索結果顯示，它不像 **binary encoding**、**one-hot encoding**、**domain-wall encoding** 那樣，在組合最佳化社群中有一個唯一標準定義。最接近本研究需要的是「Boolean hypercube representation」；另外一個明確命名的技術則是 HyperNEAT 的 A Hypercube-Based Encoding for Evolving Large-Scale Neural Networks，兩者概念相關，但用途不同。³

對組合最佳化最自然的定義為：

$$Q_n = (V, E), \quad V = \{0, 1\}^n, \\ (x, y) \in E \iff d_H(x, y) = 1,$$

其中

$$d_H(x, y) = \sum_{i=1}^n 1[x_i \neq y_i] =$$

為 Hamming distance。也就是說，每一個二進位解是一個 n -dimensional hypercube vertex，一次 bit flip 是超立方體上的一條邊。Boolean-hypercube polynomial optimization 的原始研究也直接把 feasible domain 寫成 $\mathbf{x} \in \{a_i, b_i\}^n$ 。³²

這個結構與 HH 的關係比表面上更強。假定目前解為 x ，選出一組允許重新最佳化的座標 $S \subseteq \{1, \dots, n\}$ ，則

$$\mathcal{C}_S(x) = \{y \in \{0, 1\}^n : y_i = x_i, \forall i \notin S\}$$

就是一個維度 $|S|$ 的 subcube，候選數只有

$$|\mathcal{C}_S(x)| = 2^{|S|}.$$

因此 HH 不需要把整個 n -variable 問題送進量子電腦；它只需學習「下一次要解哪個 k -dimensional subcube」。這相當於一種 quantum-ready large-neighborhood search： k 越小，需要的 logical qubits 越少； k 越大，探索能力越強但量子資源與 encoding cost 越高。這是本報告提出的整合方式，數學上建立於 Boolean hypercube 與 QUBO representation。³³

Hypercube／相關 encoding 的證據整理

概念	數學／編碼方式	應用	已有結果	與 HH／量子整合的價值	證據
Boolean hypercube	$x \in \{0, 1\}^n$, Hamming-1 為鄰接	pseudo-Boolean、 SAT、QUBO、 binary combinatorial optimization	理論上許多 discrete problems 可寫 成 Boolean- hypercube polynomial optimization	最適合定義 HH 的 bit-flip、 k - flip、subcube neighborhoods	Dressler, Kurpisz, de Wolff ; arXiv:1802.10004 ³²
SONC optimization over hypercube	$\min f(x),$ $p_j(x) \geq 0,$ $x \in \{a_i, b_i\}^n$	polynomial combinatorial optimization	對 degree- d constrained polynomial，其 研究建立 hypercube 上 SONC certificate 的 degree upper bound $n + d$ ， 並證明短 certificate 存在 性	提供 hypercube 不 只是幾何比喻， 而是正式離散最 佳化 domain 的 理論依據	MFCS / arXiv ³⁴
HyperNEAT hypercube-based indirect encoding	將幾何座標放 入 hypercube， 由 CPPN 間接 決定網路 connectivity/ weight	large-scale neural- network neuroevolution	證明間接編碼可 利用幾何規律產 生大規模 connectivity pattern	可啟發「生成 heuristic structure」或 learned encoding，但 不是 QUBO hypercube	Stanley, D'Ambrosio, Gauci ; DOI 10.1162/artl.2009.15.2.15202 ³⁵
SAT / binary CSP 的 hypercube interpretation	每一 truth assignment 是 $\{0, 1\}^n$ vertex；單變 數 flip 是 edge	SAT/local search	這是 Boolean solution-space 的直接結果，而 不是一個獨立 「Hypercubic SAT encoder」 演算法	非常適合定義 HH move operators 與 Hamming neighborhoods	Boolean- hypercube optimization framework ³²
Permutation／routing 的 constrained hypercube	例如 one-hot $x_{i,t}$ ，再加 $\sum_i x_{i,t} =$ $1, \sum_t x_{i,t} =$ 1	TSP、 routing、 sequencing	全部 binary vectors 只有極 少部分是 feasible permutations， 因此 penalty／ repair 必要	適合 quantum QUBO，但 「feasible- vertex density」可能 很低	Qiskit Optimization 提供 QP/QUBO 與 vehicle-routing 範 例 ³⁶

概念	數學／編碼方式	應用	已有結果	與 HH／量子整合的價值	證據
Domain-wall encoding	離散 variable 由 1D Ising chain 的 domain-wall position 表示	scheduling、colouring、一般 categorical decision	Chancellor 所測 scheduling/ colouring embedding 中與 one-hot 相當或更好，但問題依賴明顯	可把 encoding 本身變成 HH action	DOI 10.1088/2058-9565/ab33c2 ⁴
Path planning 的 hypercube mapping	可將 edge selection、cell decisions 或 discretized control 表成 binary vector	route/path planning	本次檢索未找到近五年以「 hypercubic encoding 」為核心名稱、且具代表性實驗的 path-planning 研究線	可作新實驗，但不應宣稱已有成熟證據	本次 structured search 結果；Boolean-hypercube 理論參考 ³²

因此，在論文命名上，本報告不建議直接宣稱「提出一種 hypercubic encoding」而未進一步定義。更精確的術語是：

Boolean-hypercube neighborhood encoding、hypercube-constrained search representation，或 adaptive subcube encoding for quantum-assisted hyper-heuristics。

這能清楚區別於 HyperNEAT，也能讓審稿者直接看到數學結構。

對排程而言，特別值得比較三種表示。假定一個離散決策有 K 個值：

$$\begin{aligned} \text{binary: } & \lceil \log_2 K \rceil \text{ bits,} \\ \text{one-hot: } & K \text{ binary variables,} \end{aligned}$$

而 domain-wall 通常利用一條 Ising chain 中 domain wall 的位置表示 K 個離散值，以約 $K - 1$ 個可變 spins/qubits 加固定邊界構造；其優點不只是少一個 qubit，而是 interaction/constraint structure 可能較容易嵌入特定 QA topology。Chancellor 的結果同時也警告，不能單靠 qubit count 判斷 encoding 優劣。 ⁴

binary encoding 雖然 qubit 數最少，但若 categorical constraint 需要複雜高階 polynomial，再 quadraticize 成 QUBO，可能產生 ancilla variables 或密集 couplings；one-hot 雖使用較多變數，constraint 結構卻簡單。因此最佳 encoding 是一個 **instance-dependent algorithm-selection problem**。這正是 HH 最有理由存在的地方。Qiskit Optimization 能把 high-level quadratic program 轉換為量子最佳化 representation，而 D-Wave Ocean 則在 BQM/CQM/QUBO 及 embedding 上提供相對完整的實驗工具鏈。 ³⁷

三者整合架構與實驗路線圖

本報告最推薦的架構稱為 **Quantum-Aware Hypercube Hyper-Heuristic (QA-HH)**。它不要求 quantum solver 在全域上擊敗 classical SOTA，而是讓 quantum method 與 classical heuristic 在同一 low-level pool 競爭。高階政策觀察 instance/search/hardware context 後，決定：

$$a_t = (\text{solver}, \text{encoding}, \text{subcube}, \text{parameters}).$$

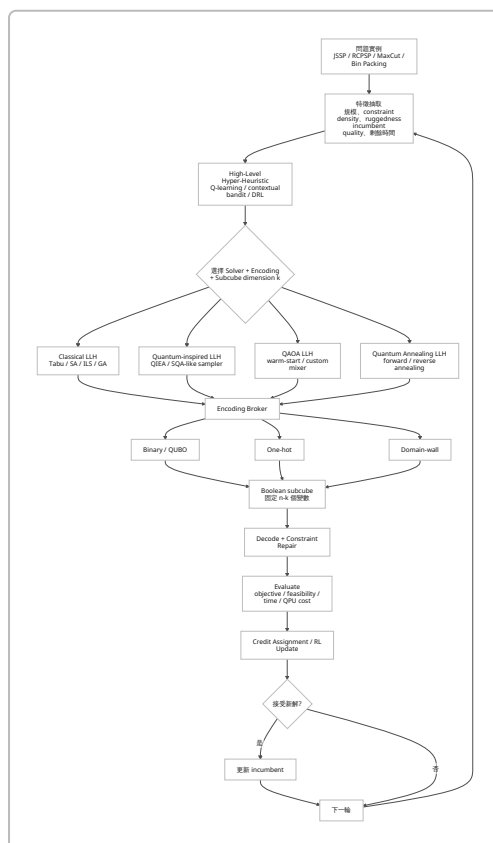
例如：

$$a_t = (\text{QAOA}, \text{domain-wall}, S_t, p = 2)$$

或

$$a_t = (\text{Tabu}, \text{binary}, S_t, \text{tenure} = 7).$$

這比單純學「下一個使用 swap 還是 insertion」更一般化：**solver class**、**encoding**、**neighborhood dimension**、**quantum resource allocation** 全部都變成 HH action。Warm-start QAOA、reverse annealing 與 Ocean/Qiskit 的 hybrid interfaces 都使此架構具備實作基礎。 38



這個架構的關鍵並非量子方塊，而是 **Encoding Broker + Subcube Decomposer**。例如一個原始 JSSP QUBO 需要 $n = 600$ logical variables，但當前解只有 20 個 operation decisions 被選入 neighborhood S_t 時，其餘變數固定，只把局部子問題送給 QAOA/QA。這並不保證量子優勢，但可以顯著把研究問題從「600-variable 全域問題」轉成「20-variable quantum local improvement」；類似 decomposition/warm-start 的必要性也可從 QAOA depth 限制與 D-Wave scheduling 的 embedding 膨脹看出。 39

另一個更具新穎性的方向是**不要用量子 solver 解原始 scheduling solution，而是解高階 heuristic sequence**。若有 m 個 low-level heuristics，在第 t 個決策點定義

$$z_{t,h} \in \{0, 1\}, \quad \sum_{h=1}^m z_{t,h} = 1.$$

可設計預測 reward $\hat{r}_{t,h}$ 與切換成本 $c_{h,h'}$ ：

$$\min_z - \sum_{t,h} \hat{r}_{t,h} z_{t,h} + \lambda \sum_{t,h,h'} c_{h,h'} z_{t,h} z_{t+1,h'} + P \sum_t \left(\sum_h z_{t,h} - 1 \right)^2.$$

這就是標準 QUBO 形式，可交給 QAOA 或 QA；也就是用量子最佳化來選 **heuristic sequence**，而不是直接解 JSSP。QUBO 與 QAOA/QA 的適配性由 Qiskit 與 D-Wave 官方工具直接支持。 40

建議的實驗設計

主實驗可採四層 benchmark。第一層使用 **MaxCut**，因為 QAOA 文獻成熟且 optimal/near-optimal solution 容易比較；warm-start QAOA 已有超過千個小圖實驗作參照。第二層為 **JSSP**，直接對照 2022 QA 與 2023 QAOA 文獻。第三層為 **RCPSP/PSPLIB**，因為 2024 D-Wave RCPSP 論文已使用 PSPLIB 脈絡並建立 QUBO formulation。第四層加入 bin packing，驗證 quantum-aware HH 能否跨不同 problem families generalize。 41

建議驗證以下假設：

假設	實驗比較	成功條件
H1：adaptive solver selection 有效	Quantum-aware HH vs classical-only HH vs fixed quantum solver	在相同 wall-clock / evaluation budget 下，平均 optimality gap 顯著較低
H2：subcube decomposition 可改善量子可擴展性	full QUBO vs $k = 8, 12, 16, 20, \dots$ subcube	physical/logical qubits、embedding cost 降低，同時保留顯著 improvement probability
H3：encoding selection 應 instance-dependent	binary vs one-hot vs domain-wall vs HH-adaptive encoding	adaptive policy 的 aggregate rank 優於任何單一 encoding
H4：warm-start > cold-start quantum LLH	cold QAOA/QA vs incumbent-seeded warm/reverse variant	success probability、best-of-budget gap 改善
H5：quantum call 不應每一步都使用	always-quantum vs learned invocation policy	learned policy 以較少 QPU calls 達到相同或更佳 solution quality
H6：高階 heuristic-sequence QUBO 比 full-problem QUBO 更容易擴展	QAOA/QA 選 heuristic sequence vs QAOA/QA 直接解排程	更少 qubits、較高 feasibility rate，且 end-to-end objective 有改善

統計上建議每個 instance/algorithm 至少多次獨立重跑；除了 mean/std 之外，使用 median、bootstrap confidence interval、Friedman/Wilcoxon 類 non-parametric comparison 與 effect size。這是本報告的實驗設計建議，而不是對既有文獻結果的描述。

核心效能指標不應只看 objective。對 quantum-hybrid HH，至少同時記錄：

$$\text{Gap} = \frac{f - f^*}{|f^*|} \times 100\%,$$

以及 best-so-far curve、success probability P_{opt} 、constraint-feasibility rate、time-to-target、wall-clock time、QPU execution/access time、quantum calls、logical qubits、physical qubits、embedding ratio、chain-break fraction、QAOA circuit depth、two-qubit gate count、shots、repair rate。D-Wave 的 JSSP 文獻特別分析 chain breaks，而 RCPSP 工作使用 time-to-target / Q-score，說明只報「annealing microseconds」而忽略 embedding 與前後處理會是不公平的比較。 31

對 hypercube representation，再增加三個專門指標：

$$\rho_F = \frac{\#\{\text{feasible vertices}\}}{2^k},$$

即 subcube feasible density；

$$L = \text{corr}(d_H(x, y), |f(x) - f(y)|),$$

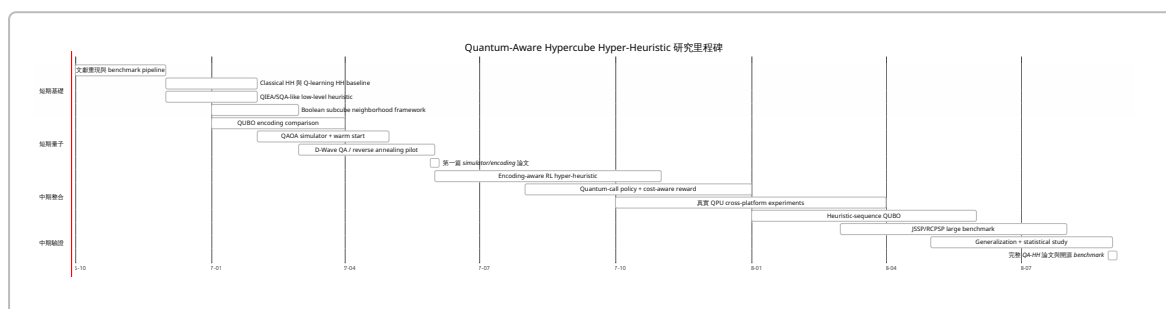
用於衡量 Hamming locality 是否對 objective 有意義；以及 **repair rate / repair cost**。如果一個 encoding 使 Hamming-1 鄰居幾乎全部 infeasible，那麼它雖然「數學上是 hypercube」，對 HH 卻不是好的 neighborhood representation。

風險與緩解策略如下：

風險	為何重要	緩解
Qubit explosion	time-indexed JSSP/RCPSP 容易大量增加 binary variables	subcube decomposition、fixing、rolling horizon、problem-specific compact encoding
Minor-embedding overhead	logical interaction graph 可能遠比 QPU topology 稠密	domain-wall / sparse encoding；先用 embedding estimator 篩 solver
Chain breaks	QA physical chains 不一致會破壞 logical solution	chain-strength tuning、embedding-aware HH features、postprocessing
QAOA noise/depth	p 增加提高電路深度	warm-start、 $p = 1 \sim 3$ 起步、recursive/decomposed QAOA
Classical optimizer overhead	QAOA 外迴圈可能需要大量 circuit evaluations	parameter transfer、warm start、固定 schedule、bandit-based parameter selection
Constraint penalties	penalty 太低產生 infeasible；太高壓縮有效 energy scale	adaptive penalty calibration；constraint-preserving mixer/domain-wall
QPU latency/queue	end-to-end time 可能遠超 quantum kernel time	將 QPU 呼叫視為昂貴 action；batching；訓練階段主要用 simulator
Quantum-inspired 被誤稱 speedup	QIEA/SQA 若在 CPU 執行並非 quantum advantage	全文分開報「quantum hardware」與「quantum-inspired」
Hypercube locality 不佳	bit distance 不一定代表 schedule distance	learned encoding、operator-aware Hamming mapping、repair/decoder
Overfitting instances	RL-HH 可能只記住 training distributions	train/test problem-family split、unseen size/generalization testing

QA JSSP 所見的 chain-break 與 qubit scaling、QAOA 官方文件對 depth 的實務考量，以及 domain-wall 對 embedding structure 的敏感性，都直接支持上述風險設計。 ⁴²

建議時間線



6–12 個月的合理成果目標不是「證明 quantum speedup」，而是建立一個 reproducible framework：classical HH + quantum-inspired + QAOA simulator + QA interface + binary/one-hot/domain-wall/subcube encoding comparison。這個階段即使結果顯示 quantum LLH 沒有提升，仍可產生有價值的 encoding/algorithm-selection negative result。

1–2 年才應進入 encoding-aware policy、真實 QPU、reverse annealing、warm-start QAOA、cross-problem generalization 與 heuristic-sequence QUBO。這樣的時間順序也能避免研究第一年就被硬體 availability 或 qubit limit 綁死。

工具平台、結論與建議

目前軟體生態已足以在沒有自建量子硬體的情況下完成前兩階段研究。特別是 Qiskit Optimization、D-Wave Ocean、PennyLane 與 Cirq 都提供 Python interface；其中 Qiskit Optimization 專注 high-level optimization model → quantum optimizer，D-Wave Ocean 最適合 QUBO/BQM/QA，PennyLane 適合 differentiable hybrid experiments，Cirq 則適合細部電路、noise 與 hardware-aware QAOA 實驗。⁴³

工具／平台	最適用途	本研究適合度	優點	注意事項	官方來源
Qiskit + Qiskit Optimization 0.7	QAOA、warm-start、recursive QAOA、QUBO/QP	★★★★★	high-level QuadraticProgram、轉換工具、可接 simulator/IBM hardware	gate-model noise、shot cost、depth	Qiskit Optimization 44
Qiskit Warm-Start QAOA	incumbent → QAOA	★★★★★	與 proposed HH architecture 幾乎直接對應	warm-start quality 仍影響結果	官方 tutorial 45
D-Wave Ocean SDK 9.5	QA、reverse QA、BQM/QUBO、hybrid solver	★★★★★	QPU、classical sampler、hybrid solver 共用 workflow；minor embedding 工具完整	QPU connectivity、chain strength、embedding overhead	Ocean SDK 46

工具／平台	最適用途	本研究適合度	優點	注意事項	官方來源
PennyLane	hybrid variational quantum algorithm、QAOA prototype	★★★★☆	automatic differentiation、跨 simulator/hardware backend	組合最佳化 modeling 沒有 Qiskit Optimization/Ocean 那麼專門	PennyLane devices ⁴⁷
PennyLane default.qubit / lightning.qubit	本地 QAOA 模擬	★★★★☆	很適合原型與 gradient experiments	state-vector memory 指數成長	default.qubit ⁴⁸
Cirq	circuit-level QAOA、noise、hardware-aware experiments	★★★★☆	circuit manipulation、parameterized circuit、noise/device model	optimization layer需自行建立較多	Google Quantum AI Cirq ⁴⁹
D-Wave classical/hybrid samplers	SA baseline、QA ablation、hybrid	★★★★★	與 QPU 使用相同 BQM abstraction，可做公平 pipeline	hybrid solver 不等於 pure quantum algorithm	Ocean software stack ⁵⁰
Google OR-Tools CP-SAT	排程 classical baseline / optimality reference	★★★★★	JSSP modeling 成熟，非常適合作為 classical comparator	非 HH / 非量子	官方 JSSP 範例 ⁵¹
CustomHys	continuous HH / automated metaheuristic design	★★★☆☆	可參考 HH operator-composition architecture	需自行擴充至 scheduling + quantum LLH	Cruz-Duarte 團隊開源實作 ⁵²

實作語言建議統一採 **Python**。這不是因為 Python 本身在演算法上更快，而是 Qiskit、Cirq、PennyLane、Ocean 與 OR-Tools 都能在同一 Python experiment orchestration layer 中整合，因而最適合進行跨 solver、跨 encoding 的公平 benchmark。Cirq 官方定位本身就是 Python quantum-circuit library；Ocean 目前亦以 Python SDK 為主要介面。 ⁵³

硬體上，第一年不需要購買量子硬體。建議配置為 multicore CPU、64–128 GB RAM，視較大 state-vector simulation 再加入 GPU；大部分 HH、QIEA、QUBO generation、classical baselines 先本地執行。真正需要真實硬體時，使用 IBM-compatible gate QPU 與 D-Wave Leap/Advantage 類平台；D-Wave Ocean 本身就設計為 CPU、QPU 與 quantum-classical hybrid solver 可互換的工具鏈。 ⁵⁴

是否值得投入？答案是「值得，但研究命題必須選對」。不建議把主要假設設成「QAOA 或 QA 一定比 classical HH 快」。目前 JSSP、RCPSp、MaxCut 等研究並沒有提供足以支持如此一般化結論的證據，而且 QAOA depth/noise、QA embedding/chain breaks、QUBO penalty 與 logical/physical qubit expansion 都仍是實際障礙。 ⁵⁵

真正值得投入的第一優先研究題目是：

Encoding- and Resource-Aware Quantum Hyper-Heuristic with Adaptive Boolean-Hypercube Neighborhoods

其核心 novelty 可以明確分成三層：

第一層是 solver selection。高階 HH 不只選 swap/insertion 等 heuristics，而是在 Tabu、SA、QIEA、SQA-like sampler、QAOA、QA/reverse QA 之間選擇。

第二層是 encoding selection。高階控制器同時決定 binary、one-hot、domain-wall 或其他 problem-specific representation。Domain-wall 既有研究已顯示，encoding 的好壞具有問題依賴性，因此這不是無意義的額外自由度。⁴

第三層是 quantum-resource selection。Policy 決定 k -dimensional subcube、QAOA p 、shots、QA anneal schedule 或是否完全不呼叫 QPU。這讓硬體限制不再只是 nuisance，而成為 HH state/action space 的一部分。Qiskit 的 recursive/warm-start 最佳化與 D-Wave hybrid stack 都提供實作上的支撐。⁵⁶

若以論文貢獻角度排序，本報告建議：

研究題目	新穎度	風險	建議
Adaptive hypercube neighborhood HH	高	低	第一優先
Encoding-aware HH : binary / one-hot / domain-wall	很高	低至中	第一優先
Quantum-inspired HH	中高	低	適合作為第一篇與基線
QA/reverse-QA low-level heuristic	很高	中	第二優先
Warm-start / recursive QAOA LLH	很高	中高	第二優先
Quantum optimization of heuristic sequence	極高	中高	適合中期論文
Full JSSP/RCPSP quantum solver	中	很高	不建議作主題核心
宣稱 quantum speedup	高但風險極高	極高	現階段不建議

最有可能產生實際影響的成果，未必是「量子演算法平均贏過所有經典演算法」，而可能是找到一條 **phase diagram / decision rule**：例如「constraint density 低、subcube $k \leq 20$ 、incumbent quality 高時 reverse QA 有效；另一種結構下 tabu search 較好；某類 categorical subproblem domain-wall embedding 優於 one-hot」。這正是 hyper-heuristic 的強項：把『哪個演算法最好』改寫成『什麼情境該用哪個演算法』。Domain-wall encoding 的 problem-dependence、QAOA warm-start 的效果以及 QA scheduling 的硬體限制，都使這種研究問題比追求單一 universal quantum solver 更有根據。⁵⁷

就目前文獻狀態而言，三者交叉最大的學術空缺可濃縮成一句話：

Hyper-Heuristic + Adaptive Encoding + Quantum/Classical Solver Selection + Boolean Subcube Decomposition

這一架構同時承接近五年 HH 的 learning-based algorithm selection 趨勢、量子最佳化的 warm-start/decomposition 趨勢，以及 Boolean hypercube 對 binary combinatorial spaces 的自然表示。⁵⁸

主要引用與原始來源連結

類別	來源
HH 綜述	Dokeroglu, Kucukyilmaz & Talbi, Hyper-heuristics: A survey and taxonomy, C&IE 187, 109815 — DOI ¹¹
RL-HH 綜述	Li et al., A review of reinforcement learning based hyper-heuristics, PeerJ Computer Science 10:e2141 — DOI ¹²
Multi-objective HH	Systematic literature review of multi-objective hyper-heuristics — DOI ¹⁹
Automated metaheuristic design	Cruz-Duarte et al., Hyper-Heuristics to customise metaheuristics for continuous optimisation — DOI ⁵⁹
RL algorithm selection	Gölcük & Ozsoydan, Q-learning and hyper-heuristic based algorithm recommendation for changing environments — DOI ¹⁴
Surrogate HH	Zhong et al., Surrogate Ensemble-Assisted Hyper-Heuristic Algorithm for Expensive Optimization Problems — DOI ¹⁶
Hierarchical HH	Guerriero & Saccomanno, A hierarchical hyper-heuristic for the bin packing problem — DOI ¹⁵
Workflow HH	Zaki et al., cooperative coevolutionary GP-HH — DOI ⁶⁰
LLM HH	Zhong et al., LLMOA — DOI ⁶¹
QAOA 原始方法	Farhi, Goldstone & Gutmann, A Quantum Approximate Optimization Algorithm — arXiv:1411.4028 ⁶²
QAOA JSSP	Kurowski et al., Application of quantum approximate optimization algorithm to job shop scheduling problem — DOI ⁶³
Warm-start QAOA	Tate et al., Warm-Started QAOA with Custom Mixers... — DOI ²⁵
Iterative QAOA JSSP	Lopez-Ruiz et al., A Non-Variational Quantum Approach to the Job Shop Scheduling Problem — arXiv:2510.26859 ⁶⁴
QA JSSP	Carugno, Ferrari Dacrema & Cremonesi, Evaluating the job shop scheduling problem on a D-Wave quantum annealer — DOI ²⁷
QA RCPSP	Pérez Armas, Creemers & Deleplanque, Solving the resource constrained project scheduling problem with quantum annealing — DOI ⁶⁵
Domain-wall encoding	Chancellor, Domain wall encoding of discrete variables for quantum annealing and QAOA — DOI ⁴
Boolean hypercube	Dressler, Kurpisz & de Wolff, Optimization over the Boolean Hypercube via Sums of Nonnegative Circuit Polynomials — arXiv:1802.10004 ³²
HyperNEAT	Stanley, D'Ambrosio & Gauci, A Hypercube-Based Encoding for Evolving Large-Scale Neural Networks — DOI ³⁵
IBM QAOA	IBM Quantum, QAOA tutorial — 官方文件 ⁶⁶
Qiskit Optimization	官方文件 ⁶⁷

類別	來源
Qiskit Warm Start	官方 tutorial 45
D-Wave Ocean	官方 Ocean SDK 68
D-Wave software stack	官方文件 54
Google Cirq	Google Quantum AI 官方文件 69
PennyLane	官方 devices / simulators 47
OR-Tools JSSP	Google OR-Tools 官方 JSSP 51

最終建議：值得投入，而且應優先投入「量子感知的超啟發式」而非「純量子求解器」。短期以 quantum-inspired + hypercube subspace + encoding-selection 建立可重現結果；第二階段再加入 D-Wave reverse annealing；第三階段才加入 warm-start/recursive QAOA 真實 QPU。這條路徑的優點是，即使最終沒有觀察到量子優越性，研究仍能產出具有獨立價值的 **encoding-selection**、**adaptive decomposition**、**solver-selection** 與公平 quantum/classical benchmarking 結果；若特定 problem regime 確實出現量子低階 heuristic 的優勢，HH 又能自然學會只在那些 regime 呼叫量子資源。這比把整個研究成敗押在「量子演算法全面擊敗經典演算法」上，科學風險低得多，也更符合截至 2026 年的實證文獻。 70

- 1 6 9 11 20 <https://pure.eur.nl/en/publications/hyper-heuristics-a-survey-and-taxonomy/>
<https://pure.eur.nl/en/publications/hyper-heuristics-a-survey-and-taxonomy/>
- 2 27 31 42 55 70 <https://www.nature.com/articles/s41598-022-10169-0>
<https://www.nature.com/articles/s41598-022-10169-0>
- 3 35 <https://dl.acm.org/doi/abs/10.1162/artl.2009.15.2.15202>
<https://dl.acm.org/doi/abs/10.1162/artl.2009.15.2.15202>
- 4 57 <https://arxiv.org/abs/1903.05068>
<https://arxiv.org/abs/1903.05068>
- 5 8 66 <https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm>
<https://quantum.cloud.ibm.com/docs/en/tutorials/quantum-approximate-optimization-algorithm>
- 7 22 62 <https://arxiv.org/abs/1411.4028>
<https://arxiv.org/abs/1411.4028>
- 10 13 59 <https://www.sciencedirect.com/science/article/pii/S2210650221000961>
<https://www.sciencedirect.com/science/article/pii/S2210650221000961>
- 12 58 <https://pubmed.ncbi.nlm.nih.gov/38983203/>
<https://pubmed.ncbi.nlm.nih.gov/38983203/>
- 14 <https://www.sciencedirect.com/science/article/abs/pii/S0952197621001317>
<https://www.sciencedirect.com/science/article/abs/pii/S0952197621001317>
- 15 <https://link.springer.com/article/10.1007/s00500-022-07118-4>
<https://link.springer.com/article/10.1007/s00500-022-07118-4>
- 16 <https://link.springer.com/article/10.1007/s44196-023-00346-y>
<https://link.springer.com/article/10.1007/s44196-023-00346-y>

17 60 <https://www.sciencedirect.com/science/article/pii/S0305054824002776>
<https://www.sciencedirect.com/science/article/pii/S0305054824002776>

18 61 <https://www.sciencedirect.com/science/article/abs/pii/S1474034624006931>
<https://www.sciencedirect.com/science/article/abs/pii/S1474034624006931>

19 <https://link.springer.com/article/10.1007/s10462-026-11531-8>
<https://link.springer.com/article/10.1007/s10462-026-11531-8>

21 24 63 <https://www.sciencedirect.com/science/article/pii/S0377221723002072>
<https://www.sciencedirect.com/science/article/pii/S0377221723002072>

23 <https://arxiv.org/html/2409.05542v3>
<https://arxiv.org/html/2409.05542v3>

25 30 38 41 <https://quantum-journal.org/papers/q-2023-09-26-1121/>
<https://quantum-journal.org/papers/q-2023-09-26-1121/>

26 64 <https://arxiv.org/abs/2510.26859>
<https://arxiv.org/abs/2510.26859>

28 65 <https://www.dwavequantum.com/resources/publication/solving-the-resource-constrained-project-scheduling-problem-with-quantum-annealing/>
<https://www.dwavequantum.com/resources/publication/solving-the-resource-constrained-project-scheduling-problem-with-quantum-annealing/>

29 <https://link.springer.com/article/10.1007/s00170-009-2438-4>
<https://link.springer.com/article/10.1007/s00170-009-2438-4>

32 33 <https://arxiv.org/html/1802.10004v1>
<https://arxiv.org/html/1802.10004v1>

34 <https://arxiv.org/abs/1802.10004>
<https://arxiv.org/abs/1802.10004>

36 https://qiskit-community.github.io/qiskit-optimization/tutorials/07_examples_vehicle_routing.html
https://qiskit-community.github.io/qiskit-optimization/tutorials/07_examples_vehicle_routing.html

37 40 43 67 https://qiskit-community.github.io/qiskit-optimization/apidocs/qiskit_optimization.html
https://qiskit-community.github.io/qiskit-optimization/apidocs/qiskit_optimization.html

39 https://qiskit-community.github.io/qiskit-optimization/tutorials/03_minimum_eigen_optimizer.html
https://qiskit-community.github.io/qiskit-optimization/tutorials/03_minimum_eigen_optimizer.html

44 https://qiskit-community.github.io/qiskit-optimization/getting_started.html
https://qiskit-community.github.io/qiskit-optimization/getting_started.html

45 56 https://qiskit-community.github.io/qiskit-optimization/tutorials/10_warm_start_qaoa.html
https://qiskit-community.github.io/qiskit-optimization/tutorials/10_warm_start_qaoa.html

46 68 <https://docs.dwavequantum.com/en/latest/ocean/index.html>
<https://docs.dwavequantum.com/en/latest/ocean/index.html>

47 <https://pennylane.ai/devices>
<https://pennylane.ai/devices>

48 <https://pennylane.ai/devices/default-qubit>
<https://pennylane.ai/devices/default-qubit>

49 53 69 <https://quantumai.google/cirq>

<https://quantumai.google/cirq>

50 54 <https://docs.dwavequantum.com/en/latest/ocean/stack.html>

<https://docs.dwavequantum.com/en/latest/ocean/stack.html>

51 https://developers.google.com/optimization/scheduling/job_shop

https://developers.google.com/optimization/scheduling/job_shop

52 <https://github.com/jcrvz/customhys>

<https://github.com/jcrvz/customhys>